FITNESS PRO

Cloud Taşıma ve Canlıya Geçiş Rehberi

Linux VPS • Docker Compose • .NET 10 API • SQL Server konteyneri • Frontend

Amaç: mevcut Fitness Pro mimarisini koruyarak kontrollü, güvenli, yedeklenebilir ve geri alınabilir bir üretim ortamına taşımak.

Hazırlanma tarihi: 21 Ekim 2026

Alan adı henüz alınmadığı ve sunucu sağlayıcısı kesinleşmediği için sağlayıcıya özgü fiyatlar, IP adresleri, parolalar ve alan adları özellikle uydurulmamıştır. Bunlar yer tutucularla gösterilmiştir.

ÖNEMLİ: Bu belge bir uygulama planıdır. Komutları çalıştırmadan önce sağlayıcının güncel şartlarını, işletim sistemi sürümünü, SQL Server lisansını, yedekleri ve projenin gerçek yapılandırmasını doğrulayın.

1. Belgenin amacı, kapsamı ve çalışma kuralları

Bu rehber, Fitness Pro'nun mevcut uygulamasını yeniden tasarlamadan Linux VPS üzerinde çalıştırmak için hazırlanmıştır. Hedef düzen; bir Linux VPS, Docker Engine ve Docker Compose, .NET 10 API konteyneri, aynı sunucuda özel Docker ağı içinde çalışan SQL Server konteyneri ve mevcut frontend'dir. Frontend'in nerede barındırıldığına ilişkin kesin karar verilmediğinden iki seçenek açıklanır: frontend'i mevcut statik barındırma yerinde bırakmak veya aynı VPS üzerinde reverse proxy ile sunmak.

Bilinen proje bağlamı: API daha önce Docker içinde 8080 portunda, ana makinede 8081 portuna eşlenmiş olarak çalıştırılmış; veritabanı sunucusu için fitnesspro-db benzeri bir konteyner adı kullanılmış; sağlık denetimi için /health uç noktası daha önce test edilmiştir. Bunlar geçmiş geliştirme ortamına ait ipuçlarıdır; canlı ortamda mevcut compose dosyası, connection string, DB adı, port ve health endpoint mutlaka yeniden kontrol edilmelidir.

Bu rehberdeki değiştirilebilir değerler

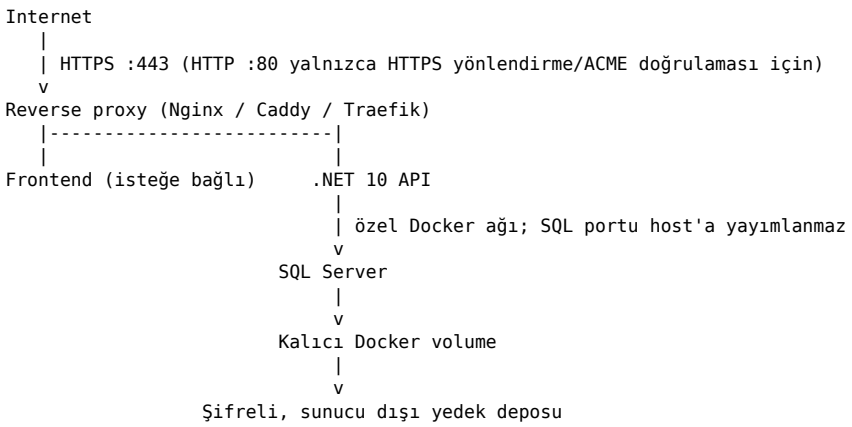
Yer tutucu	Gerçek değer nereden bulunacak?
	Hosting sağlayıcısının atadığı genel IPv4/IPv6 adresi.
	Satın alınacak alan adı; ör. gerçek alan adınız. Bu örneği kopyalamayın.
	Mevcut bağlantı ayarları ve SQL Server içinde doğrulanacak veritabanı adı.
	Uygulamanın kullanacağı, gerekli izinlerle sınırlandırılmış SQL login'i.
	Güçlü, benzersiz parola; belgeye veya Git deposuna yazılmaz.
	CI/build çıktısındaki sabit API imaj etiketi veya commit SHA.
	Frontend'in üretim ortamında kullanacağı HTTPS API adresi.

Önerilen uygulama sırası

- Mevcut sistemi envanterle; veritabanı yedeğini al ve yedeği doğrula.
- VPS'i hazırla; SSH, güncellemeler, firewall ve Docker kurulumunu tamamla.
- SQL Server'ı dışarı açmadan, kalıcı volume ile başlat; veritabanını geri yükle.
- API imajını oluştur; gizli ayarları environment/secrets üzerinden ver.
- Reverse proxy ve HTTPS kur; domain henüz yoksa geçici IP üzerinden sadece kontrollü test yap.
- Uçtan uca testleri ve yedek geri yükleme provasını yap; sonra DNS'i canlıya geçir.
- İlk gün logları izle; geri dönüş koşulları ve sorumlular net olmadan eski ortamı kapatma.

2. Mimari ve kararlar

2.1 Hedef mimari



Reverse proxy, internete açılan tek uygulama giriş noktası olmalıdır. SQL Server portu 1433, API'nin iç portu ve Docker yönetim soketi internete açılmamalıdır. SSH sadece yönetim için gerekir; mümkünse yalnızca güvenilir IP aralığına izin verin.

2.2 Tek VPS yaklaşımının artıları ve riskleri

- Artı: API ve veritabanı aynı makinede olduğu için başlangıç maliyeti ve ağ gecikmesi düşüktür.
- Eksi: VPS arızası API ve veritabanını aynı anda etkiler; bu mimari yüksek erişilebilirlik sağlamaz.
- Eksi: CPU, RAM, disk ve I/O kaynakları API, SQL Server, reverse proxy ve yedekleme tarafından paylaşılır.
- Zorunlu telafi: harici yedek, kaynak takibi, disk doluluk alarmı, güvenlik güncellemeleri ve test edilmiş geri dönüş prosedürü.

2.3 Sunucu boyutu seçimi

Gerçek yük ve veritabanı boyutu ölçülmeden kesin VPS boyutu önerilmemelidir. Küçük test/erken üretim yükü için başlangıçta 2 vCPU ve 4 GB RAM gibi düşük bir paket değerlendirilebilir; ancak SQL Server belleği, eşzamanlı istek sayısı, veritabanı boyutu ve log/backup işlemleri nedeniyle 4 vCPU ve 8 GB RAM sınıfı daha güvenli bir başlangıç olabilir. Bu rakamlar garanti veya zorunlu minimum değildir; yük testiyle doğrulanmalıdır. SSD/NVMe disk, yeterli trafik kotası, snapshot/backup seçenekleri, veri merkezi bölgesi, destek kalitesi ve yükseltme kolaylığı fiyat kadar önemlidir.

Düşük maliyet yaklaşımı: uzun dönem taahhüt vermeden önce aylık VPS ile başlayın; alan adı ve yedek depolama maliyetini ayrı hesaplayın. Sağlayıcıların güncel fiyatlarını ve KDV/kur/yenileme ücretlerini satın alma anında karşılaştırın. Ücretsiz deneme veya promosyonu üretim yedeğinin tek kopyası olarak kullanmayın.

2.4 SQL Server lisans kararı

SQL Server Developer Edition yalnızca geliştirme/test içindir; üretim iş yükü için kullanmayın. Kullanılacak SQL Server sürüm/edition ve konteyner imajı, lisans koşulları ve projenin ihtiyaçlarıyla uyumlu olmalıdır. Express/Standard/Enterprise seçeneklerini ve güncel lisans şartlarını satın almadan önce Microsoft'un resmi belgelerinden doğrulayın.

SQL Server konteynerinin Linux üzerinde çalışması, lisans yükümlülüklerini kendiliğinden ortadan kaldırmaz. Üretim için imaj etiketini belirsiz 'latest' yerine test edilmiş belirli bir sürüme sabitleyin. Mevcut veritabanının kullandığı SQL Server özelliklerini, collation'ı, compatibility level'ı ve veritabanı boyutunu önceden kontrol edin.

3. Taşıma öncesi envanter ve hazırlık

3.1 Proje envanteri

Geliştirme bilgisayarında proje köküne gidin. Gerçek klasör adları ve dosyalar yerel depoya göre değişebilir; Fronted yazımı projede bilerek bu şekilde olabilir.

```
cd C:\Projects\Fitnes-Pro
git status
git branch --show-current
git log -1 --oneline
dotnet --info
docker version
docker compose version
docker compose config
```

Amaç: hangi commit'in canlıya taşınacağını, çalışma ağacında commit edilmemiş değişiklik bulunup bulunmadığını, SDK ve Docker sürümlerini, compose yapılandırmasının çözümlenmiş halini kontrol etmek. `docker compose config` çıktısında parolalar bulunabilir; bu çıktıyı herkese açık şekilde paylaşmayın.

3.2 API ve veritabanı ayarlarını bulma

```
# PowerShell'de proje kökünde:
Get-ChildItem -Recurse -File -Include
  *.csproj,appsettings*.json,Dockerfile,compose*.yml,docker-compose*.yml
Get-ChildItem -Recurse -File -Include *.yml,*.yaml | Select-String -Pattern
  "ports:|ConnectionStrings|ASPNETCORE_URLS|fitnesspro-db"
dotnet list .\Backend\Gym-Pro-Api\Gym-Pro-Api\Gym-Pro-Api.csproj package --vulnerable
  --include-transitive
```

Yukarıdaki örnek proje yolunu doğrulayın. Gizli değerleri terminal çıktısından paylaşmadan önce maskeleyin. Üretim connection string'i, SMTP parolası, JWT imza anahtarı, token anahtarları, Google Maps anahtarı ve varsa OAuth sırları envantere alınmalı; ancak hiçbirisi PDF'ye, Git'e veya sohbet mesajına kopyalanmamalıdır.

3.3 Mevcut SQL Server'da kontrol listesi

- Gerçek DB adı, boyutu, SQL Server sürümü ve collation bilgisi.
- Tüm tablolar, view, stored procedure, trigger, index ve ilişkiler.
- Login/user eşleştirmeleri ve uygulama için gereken en düşük izinler.
- SQL Server Agent job, scheduled task, dış dosya, FILESTREAM veya linked server gibi taşımayı etkileyen özellikler.
- Identity/sequence değerleri, tarih/saat ayarları ve veritabanı uyumluluk düzeyi.
- Son başarılı yedek ve bu yedekten test ortamına geri yükleme sonucu.

3.4 Dondurma penceresi

Taşıma anında eski sistemde yeni kullanıcı, mesaj veya profil değişikliği gerçekleşirse eski ve yeni veritabanları ayrışabilir. Kesin geçişte kısa bir bakım penceresi planlayın: yazma işlemlerini durdurun veya eski API'yi salt okunur/kapalı moda alın, son yedeği alın, hedefe geri yükleyin, test edin ve DNS/uygulama trafiğini yeni sisteme yönlendirin. Süreç doğrulanmadan eski veritabanını silmeyin.

4. VPS satın alma ve ilk güvenlik kurulumu

4.1 Sağlayıcı seçim ölçütleri

- Linux Ubuntu Server LTS veya Debian Stable için resmi destek; x86-64 mimari ve SQL Server imajının desteklediği platform.
- SSD/NVMe, büyütülebilir disk, yeterli RAM, trafik kotası ve yedek/snapshot seçenekleri.
- Sunucuya konsol erişimi ve sağlayıcı firewall'ı; acil durumda kurtarma/yeniden kurulum yolu.
- Türkiye/Avrupa gecikmesi, veri yerleşimi ve KVKK kapsamındaki veriler için hukuki değerlendirme.
- Aylık toplam maliyet: VPS + alan adı + yedek depolama + e-posta gönderimi + olası lisans/KDV.

İlk kurulum için Ubuntu Server LTS yaygın bir seçenektir. Dağıtımın güncel destek durumu, Docker resmi destek listesi ve SQL Server konteynerinin destek matrisi satın alma/kurulum günü kontrol edilmelidir.

4.2 SSH anahtarı ve yönetici hesabı

Sağlayıcı panelinden SSH public key ekleyin. Özel anahtarı asla sunucuya, Git'e veya birine göndermeyin. İlk oturumdan sonra normal bir yönetici hesabı oluşturun; sudo kullanın. Root ile sürekli doğrudan giriş yerine normal kullanıcı ve SSH anahtarı tercih edin.

```
# Sunucuda, sağlayıcı konsolu veya ilk SSH oturumu üzerinden:
sudo adduser deploy
sudo usermod -aG sudo deploy

# Yerel Windows PowerShell'de anahtar oluşturma (önceden yoksa):
ssh-keygen -t ed25519 -a 64 -C "fitness-pro-deploy"

# Bağlantı (kullanıcı/IP değerlerini değiştirin):
ssh deploy@<VPS_IP>
```

Public key'in sunucudaki `~/.ssh/authorized_keys` dosyasında bulunduğunu ve yeni oturumla giriş yapabildiğinizi doğrulamadan mevcut oturumu kapatmayın. Sağlayıcının konsol kurtarma yolunu önceden

bilin.

4.3 İşletim sistemi güncellemeleri

```
sudo apt update
sudo apt full-upgrade -y
sudo apt install -y ca-certificates curl gnupg ufw unattended-upgrades fail2ban
sudo systemctl enable --now unattended-upgrades
sudo reboot
```

Yeniden başladıktan sonra SSH ile tekrar bağlanın. Kernel güncellemesi veya servis değişikliklerinden sonra servis durumunu kontrol edin.

```
uname -a
cat /etc/os-release
sudo systemctl --failed
df -h
free -h
uptime
```

4.4 SSH sertleştirme

SSH ayarlarını değiştirmeden önce anahtar ile ikinci bir oturumun çalıştığını doğrulayın. `/etc/ssh/sshd_config` veya `/etc/ssh/sshd_config.d/*.conf` dosyasında en azından aşağıdaki ilkeleri uygulayın. Dağıtımın include sırası nedeniyle gerçek etkin ayarları `sshd -T` ile kontrol edin.

```
# Örnek drop-in: /etc/ssh/sshd_config.d/60-fitness-pro.conf
PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication no
X11Forwarding no

sudo sshd -t
sudo sshd -T | grep -E 'permitrootlogin|pubkeyauthentication|passwordauthentication'
sudo systemctl reload ssh
```

Yanlış SSH ayarı kendinizi sunucunun dışında bırakabilir. Önce sağlayıcı konsolunu ve ikinci bir açık SSH oturumunu hazır tutun. SSH portunu değiştirmek tek başına güvenlik sağlamaz; anahtar, erişim kısıtlaması, güncelleme ve izleme daha önemlidir.

5. Firewall ve ağ sınırları

5.1 Temel UFW kuralları

```
# Önce SSH erişimine izin verin; aksi halde kendinizi kilitleyebilirsiniz.
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH

# Web için; reverse proxy ve HTTPS kurulduğunda:
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp

sudo ufw status verbose
sudo ufw enable
sudo ufw status numbered
```

SSH için sağlayıcının verdiği port standart 22 değilse `OpenSSH` profili yerine gerçek portu açın. Mümkünse SSH'yi yalnızca sabit yönetim IP'nizden kabul edin. Sağlayıcının cloud firewall'ında da aynı yaklaşımı uygulayın.

5.2 Docker portlarıyla ilgili kritik uyarı

Docker tarafından host'a yayımlanan portlar bazı kurulumlarda UFW kurallarını beklenmedik şekilde aşabilir. Sadece `ufw deny 1433` yazmak, Docker'ın `ports: - "1433:1433"` eşlemesini güvenli hale getirdiğini garanti etmez. En güvenli varsayılan, SQL Server için host portu hiç yayımlamamaktır. Ek firewall kontrolü gerekiyorsa Docker'ın resmi firewall belgelerini izleyerek DOCKER-USER zinciri veya sağlayıcı firewall'ı ile doğrulayın.

5.3 İnternete açık olması gerekenler

Port	Erişim	Gerekçe
22/tcp veya gerçek SSH portu	Yalnız yönetim IP'leri mümkünse	SSH yönetimi
80/tcp	İnternet	HTTP→HTTPS yönlendirme ve ACME doğrulaması
443/tcp	İnternet	HTTPS üzerinden web/API
1433/tcp	İnternete kapalı	SQL Server; yalnız özel Docker ağı
8080/tcp	İnternete kapalı	API iç portu; reverse proxy erişir
2375/2376 Docker API	İnternete kapalı	Docker yönetim API'si; uzaktan açık bırakmayın

6. Docker Engine ve Compose kurulumu

6.1 Resmî depodan kurulum

Aşağıdaki örnek Ubuntu içindir. Resmî Docker dokümantasyonundaki Ubuntu adımlarını kullanın; kurulum günü dağıtım sürümüne göre komutları yeniden kontrol edin. Kolaylık script'leri üretim sunucusunda körlemesine çalıştırılmamalıdır.

```
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}") stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
sudo docker run --rm hello-world
docker compose version
```

Docker'ın resmî kurulum rehberi: Docker Engine install

Güvenlik: Docker grubuna eklenen kullanıcı pratikte root düzeyinde yetki kazanabilir. Gerekmedikçe kullanıcıyı `docker` grubuna eklemeyin; yönetim komutlarını sudo ile çalıştırın.

6.2 Docker log ve disk politikası

Konteyner logları diski doldurabilir. JSON log sürücüsü için rotasyon sınırları belirleyin; ayrıca uygulama loglarında parola, token, doğrulama kodu ve kişisel verileri yazdırmayın.

```
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "5"
  }
}
```

Bu örnek `/etc/docker/daemon.json` içindir. Var olan dosyayı ezmeyin; JSON geçerliliğini doğrulayın, Docker servisini yeniden başlatmanın etkisini planlayın ve konteynerlerin yeni ayarı kullanması için gerektiğinde yeniden oluşturulacağını bilin.

7. Sunucu dizinleri ve gizli ayarlar

7.1 Dizin yapısı

```
sudo mkdir -p /opt/fitness-pro/{app,proxy,backups}
sudo chown -R deploy:deploy /opt/fitness-pro
sudo chmod 750 /opt/fitness-pro
cd /opt/fitness-pro
```

Bu dizin örnektir. Yedekler yalnız aynı VPS diskinde tutulmamalıdır; VPS kaybolursa uygulama ve yedek birlikte kaybolabilir. Yedeklerin ikinci bir konuma aktarılması zorunlu kabul edilmelidir.

7.2 .env dosyası

`.env` dosyasını Git'e koymayın. Üretim sırlarını sadece gerekli yöneticilerin okuyabildiği dosyada veya Docker secrets/uygun secret manager içinde tutun. `.env` dosyasının erişim izinlerini 600 yapın.

```
cd /opt/fitness-pro
umask 077
nano .env
chmod 600 .env

# .env için örnek alanlar; gerçek değerleri güvenli biçimde doldurun.
ACCEPT_EULA=Y
MSSQL_PID=<LICENSED_EDITION_OR_APPROVED_EDITION>
SA_PASSWORD=<LONG_RANDOM_UNIQUE_PASSWORD>
DB_NAME=<DB_NAME>
DB_APP_USER=<DB_USER>
DB_APP_PASSWORD=<LONG_RANDOM_UNIQUE_PASSWORD>
JWT_SIGNING_KEY=<LONG_RANDOM_SECRET>
ASPNETCORE_ENVIRONMENT=Production
ASPNETCORE_URLS=http://+:8080
```

Gerçek parolayı bu belgede, Git commit'inde veya terminal ekran görüntüsünde tutmayın. `SA_PASSWORD` adının veya desteklenen environment değişkenlerinin SQL Server imaj sürümüne göre doğrulanması gerekir. Parolalarda shell özel karakterleri ve Compose değişken genişletme kurallarına dikkat edin. Bir sır açığa çıktıysa sadece dosyadan silmek yeterli değildir; sır değiştirilmelidir.

7.3 Uygulama sırları

JWT imza anahtarı, SMTP kimlik bilgileri, 2FA e-posta ayarları ve dış servis API anahtarları üretim için ayrı ve benzersiz olmalıdır. Geliştirme anahtarlarını canlıda kullanmayın. Loglama seviyesini kontrollü tutun. Kullanıcıya dönen hata mesajları SQL ayrıntısı, stack trace veya yapılandırma sırrı içermemelidir.

8. Docker Compose: API + SQL Server + reverse proxy

8.1 Örnek compose tasarımı

Aşağıdaki dosya bir başlangıç şablonudur; mevcut Dockerfile, SQL Server sürümü, DB adı, health endpoint'i ve uygulamanın gerçekten okuduğu environment değişkenlerine göre uyarlanmalıdır. Mevcut compose dosyasını incelemeden körlemesine üzerine yazmayın.

```
# /opt/fitness-pro/compose.yml
services:
  api:
    image: <REGISTRY_OR_LOCAL_IMAGE>:<IMMUTABLE_TAG>
    restart: unless-stopped
    env_file:
      - .env
    environment:
      ASPNETCORE_ENVIRONMENT: Production
      ASPNETCORE_URLS: http://+:8080
    expose:
      - "8080"
    depends_on:
      - db
    networks:
      - app-internal
      - proxy-net
```

```
    # API portunu doğrudan internete yayımlamayın.
    # Gerekli değilse volumes eklemeyin.

  db:
    image: mcr.microsoft.com/mssql/server:<PINNED_SUPPORTED_TAG>
    restart: unless-stopped
    env_file:
      - .env
    environment:
      ACCEPT_EULA: ${ACCEPT_EULA}
      MSSQL_PID: ${MSSQL_PID}
      MSSQL_SA_PASSWORD: ${SA_PASSWORD}
    volumes:
      - sql-data:/var/opt/mssql
    networks:
      - app-internal
    # Bilerek ports: "1433:1433" YOK.
    # API aynı Docker ağı içinde db:1433 üzerinden bağlanır.

networks:
  app-internal:
    internal: true
  proxy-net:
    external: true

volumes:
  sql-data:
```

Bu örnekte reverse proxy'nin de `proxy-net` ağına bağlanması gerekir. `proxy-net` dış ağı önceden oluşturulmalıdır. Eğer proxy farklı bir tasarımla host üzerinde çalışıyorsa API'nin erişim modeli değişir; o durumda güvenli bir loopback bind veya Unix socket gibi yaklaşım ayrıca planlanmalıdır.

```
docker network create proxy-net
docker compose --env-file .env -f compose.yml config
docker compose --env-file .env -f compose.yml pull
docker compose --env-file .env -f compose.yml up -d
docker compose --env-file .env -f compose.yml ps
```

`docker compose config` çıktısı sırları açığa çıkarabilir; kaydetmeyin veya paylaşmayın. `--env-file` kullanımı Compose'un değişkenleri nasıl genişlettiğini etkiler; uygulama konteynerine `env_file` ile aktarılması ve compose değişkenlerinin genişletilmesi farklı işlevlerdir.

8.2 SQL Server imajı ve kaynak limitleri

SQL Server Linux konteynerinin resmi Microsoft imajını kullanın ve sürümü sabitleyin. `latest` etiketi kontrolsüz güncellemeye neden olabilir. Üretim öncesinde SQL Server sürümünü, edition'ı, lisansı, ARM/x86 mimarisini, bellek gereksinimini ve hedef veritabanıyla uyumunu doğrulayın. Kaynak limitlerini, SQL Server belleğini ve VPS kapasitesini birlikte planlayın.

8.3 API Dockerfile örneği

Aşağıdaki çok aşamalı örnek, projenin gerçek `.csproj` yolu ve bağımlılıklarına göre uyarlanmalıdır. Solution içindeki birden fazla proje varsa restore aşamasında ilgili `.csproj` dosyalarının kopyalanması gerekir.

```
# Dockerfile örneği — proje kökünden build etmek üzere uyarlayın
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY . .
RUN dotnet restore <PATH_TO_API_CSPROJ>
RUN dotnet publish <PATH_TO_API_CSPROJ> -c Release -o /out --no-restore

FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS final
WORKDIR /app
ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080
COPY --from=build /out .
USER $APP_UID
ENTRYPOINT ["dotnet", "<API_ASSEMBLY_NAME>.dll"]
```

`.NET 10` taban imaj etiketlerini resmi Microsoft .NET Docker belgelerinden doğrulayın. `USER $APP_UID` yalnız taban imaj ve dosya izinleriyle uyumluysa kullanılmalıdır. Projede `appsettings.Production.json` bulunuyorsa sır içermediğini doğrulayın; sırları imaj içine kopyalamayın.

8.4 .dockerignore

```
# .dockerignore örneği
.git
.vs
.vscode
**/bin
**/obj
**/node_modules
.env
.env.*
**/appsettings.*.local.json
**/*.pfx
**/*.key
**/secrets
backups
```

`.dockerignore` içindeki kalıpları gerçek build yapısına göre düzenleyin. Gerekli kaynak dosyalarını yanlışlıkla hariç bırakmadığınızı `docker build` ile kontrol edin.

9. Veritabanı yedeği: taşımanın en kritik adımı

9.1 Yedekleme stratejisi

Taşıma başlamadan önce kaynak SQL Server üzerinde tam veritabanı yedeği alın. Mümkünse yedeği hem mevcut sunucuya hem de şifreli, ayrı bir depolama alanına koyun. Yedek dosyasını oluşturmak tek başına başarı değildir; geri yükleme provası yapılmalıdır. Yedek dosyası kişisel veriler ve kimlik bilgileri içerebilir; erişimini kısıtlayın ve aktarımda TLS/SSH kullanın.

9.2 SQL Server T-SQL ile yedek alma

Aşağıdaki komutu SQL Server'da çalışan yönetici aracıyla, doğru DB adı ve izinlerle uygulayın. Yedek klasörü SQL Server hizmet hesabının yazabileceği bir konum olmalıdır.

```
BACKUP DATABASE [<DB_NAME>]
TO DISK = N'/var/opt/mssql/backup/<DB_NAME>_pre_migration.bak'
WITH COPY_ONLY, INIT, COMPRESSION, CHECKSUM, STATS = 10;
GO

RESTORE VERIFYONLY
FROM DISK = N'/var/opt/mssql/backup/<DB_NAME>_pre_migration.bak'
WITH CHECKSUM;
GO
```

SQL Server sürümü ve edition'ı `COMPRESSION` veya belirli backup seçeneklerini desteklemiyorsa komutu uyarlayın. Yedek hedef dizininin varlığını, yeterli disk alanını ve dosya izinlerini önceden doğrulayın. Docker konteyneri içindeki path ile host path aynı değildir.

9.3 Docker içindeki kaynak SQL Server'dan yedek alma örneği

```
# Önce gerçek konteyner adını doğrulayın:
docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Ports}}"

# SQL Server konteynerinde backup klasörünü oluşturun.
docker exec -it <SOURCE_SQL_CONTAINER> mkdir -p /var/opt/mssql/backup

# T-SQL'i sqlcmd veya yönetim aracından çalıştırın.
# sqlcmd'nin yolu ve mevcut olup olmadığı imaj sürümüne göre değişebilir.

# Yedeği host'a kopyalayın:
docker cp <SOURCE_SQL_CONTAINER>:/var/opt/mssql/backup/<DB_NAME>_pre_migration.bak .
```

Daha güvenli yöntem, T-SQL'i doğrulanmış SQL yönetim aracıyla çalıştırıp yedek dosyasını `docker cp` ile almak ve ardından şifreli aktarım kullanmaktır. Dosyayı genel web klasörüne koymayın.

9.4 Yedek dosyasını kontrol etme

```
# Linux'ta boyut ve SHA-256:
ls -lh <DB_NAME>_pre_migration.bak
sha256sum <DB_NAME>_pre_migration.bak

# Windows PowerShell'de:
Get-Item .\<DB_NAME>_pre_migration.bak | Select-Object Name,Length,LastWriteTime
Get-FileHash .\<DB_NAME>_pre_migration.bak -Algorithm SHA256
```

Kaynak ve hedefte hesaplanan SHA-256 değerleri dosyanın aktarım sırasında değişmediğini kontrol etmeye yardımcı olur. Bu, yedeğin mantıksal olarak sağlıklı olduğunu tek başına kanıtlamaz; `RESTORE VERIFYONLY` ve gerçek geri yükleme testi de gerekir.

9.5 Yedeği VPS'e güvenli aktarma

```
# Yerel Windows PowerShell'den:
scp .\<DB_NAME>_pre_migration.bak deploy@<VPS_IP>:/opt/fitness-pro/backups/

# Sunucuda:
cd /opt/fitness-pro/backups
chmod 600 <DB_NAME>_pre_migration.bak
sha256sum <DB_NAME>_pre_migration.bak
```

Büyük dosyalarda aktarımın tamamlandığını, boyutun ve hash'in kaynakla eşleştiğini kontrol edin. Dosya aktarımı sırasında bağlantı koparsa kısmi dosyayı tamamlanmış yedek saymayın.

9.6 SQL Server konteynerinde yedeği geri yükleme

Yedek dosyasını SQL Server konteynerinin görebileceği bir klasöre bind mount veya kontrollü `docker cp` ile yerleştirin. Önce `RESTORE FILELISTONLY` ile mantıksal data/log dosya adlarını öğrenin; sonra hedef disk konumlarına uygun `WITH MOVE` kullanın. Mantıksal dosya adlarını tahmin etmeyin.

```
RESTORE FILELISTONLY
FROM DISK = N'/var/opt/mssql/backup/<DB_NAME>_pre_migration.bak';
GO

-- FILELISTONLY sonucundaki LogicalName değerlerini kullanın:
RESTORE DATABASE [<DB_NAME>]
FROM DISK = N'/var/opt/mssql/backup/<DB_NAME>_pre_migration.bak'
WITH MOVE N'<LOGICAL_DATA_NAME>' TO N'/var/opt/mssql/data/<DB_NAME>.mdf',
     MOVE N'<LOGICAL_LOG_NAME>'  TO N'/var/opt/mssql/data/<DB_NAME>_log.ldf',
     RECOVERY, CHECKSUM, STATS = 10;
GO
```

Hedef veritabanı zaten varsa üzerine yazma seçeneği kullanmadan önce doğru sunucuda olduğunuzu ve mevcut hedefin atılabilir olduğunu iki kez doğrulayın. Üretim veritabanını yanlışlıkla ezmek geri dönüşü zorlaştırır.

9.7 Geri yükleme sonrası doğrulama

```
DBCC CHECKDB ([<DB_NAME>]) WITH NO_INFOMSGS;
GO

SELECT name, state_desc, recovery_model_desc, compatibility_level
FROM sys.databases
WHERE name = N'<DB_NAME>';
GO

SELECT TABLE_SCHEMA, TABLE_NAME
FROM [<DB_NAME>].INFORMATION_SCHEMA.TABLES
ORDER BY TABLE_SCHEMA, TABLE_NAME;
GO
```

`DBCC CHECKDB` kaynak miktarına göre zaman alabilir ve kaynak tüketir. Uygun bakım penceresinde çalıştırın. Tabloların sayısını, kritik tablo satır sayılarını ve ilişkili verileri kaynakla karşılaştırın. Hassas kullanıcı verisini raporlara dökmeyin.

9.8 SQL kullanıcıları ve en az ayrıcalık

Veritabanı yedeği her durumda sunucu login'lerini ve tüm sunucu düzeyi yapılandırmaları taşımayabilir. Uygulamanın login/user eşleşmesini ayrı kontrol edin. API için `sa` kullanmayın. Uygulama login'ine yalnızca ihtiyaç duyduğu veritabanında gerekli izinleri verin. EF/SQL kodunuz tablo oluşturma veya migration çalıştırıyorsa bunun üretimde hangi kimlikle yapılacağını ayrıca planlayın.

```
-- Yalnızca örnek; gerçek login oluşturma politikasına ve parola yönetimine göre uyarlayın.
-- Üretimde sa yerine uygulamaya özel login/user kullanın.
USE [<DB_NAME>];
GO
-- CREATE USER [<DB_USER>] FOR LOGIN [<DB_USER>];
-- İzinleri uygulamanın gerçek sorgularına göre en az ayrıcalıkla verin.
-- db_owner rolünü otomatik olarak vermeyin.
GO
```

10. API yapılandırması ve connection string

10.1 Docker içi bağlantı adresi

API ve SQL Server aynı Docker ağı içindeyse connection string sunucusu, SQL konteynerinin servis adını kullanmalıdır; örneğin `Server=db,1433` veya gerçek servis adı. `localhost` API konteynerinin kendisini ifade eder; SQL Server'ı ifade etmez. Geliştirme bilgisayarında kullanılan `localhost`, LocalDB veya host port eşlemesini üretim ortamına olduğu gibi taşımayın.

```
# Connection string fikri — gerçek değişken adını projenizde doğrulayın:
Server=db,1433;Database=<DB_NAME>;User
  Id=<DB_USER>;Password=<DB_PASSWORD>;Encrypt=True;TrustServerCertificate=False;
```

`TrustServerCertificate=True` sertifika doğrulamasını gevşetebilir. Üretimde şifreleme ve sertifika doğrulaması için SQL Server'ın sertifika yapılandırmasını doğru kurun; geçici olarak kullanılan güvenli olmayan ayarı kalıcı çözüm yapmayın. İç Docker ağına güvenmek, şifreleme ihtiyacını otomatik olarak ortadan kaldırmaz.

10.2 ASP.NET Core environment variable adlandırması

ASP.NET Core yapılandırması genellikle `ConnectionStrings:DefaultConnection` anahtarını `ConnectionStrings__DefaultConnection` environment değişkeniyle eşler. Fakat projenin kendi `IConfiguration` okuma şekli ve connection string anahtarı doğrulanmalıdır.

```
ConnectionStrings__DefaultConnection=<FULL_CONNECTION_STRING>
ASPNETCORE_ENVIRONMENT=Production
ASPNETCORE_URLS=http://+:8080
```

Connection string'i komut satırı argümanı olarak yazmak process listesinde veya shell geçmişinde açığa çıkabilir. `.env` dosyasının izinlerini kısıtlayın, secret manager kullanmayı değerlendirin ve uygulama loglarında connection string'i yazdırmayın.

10.3 Üretim ayarları kontrol listesi

- Production ortamı etkin; geliştirici hata sayfası kapalı.
- CORS yalnızca gerçek frontend origin'lerine izin veriyor; wildcard + credentials kombinasyonu kullanılmıyor.
- JWT imza anahtarı güçlü, rastgele, yalnız sunucu tarafında saklı ve rotasyon planı var.
- Refresh token uygulanıyorsa güvenli saklama, iptal/rotasyon ve süre politikası test edilmiş.
- SMTP host/port/TLS ve sağlayıcı kimlik bilgileri doğrulanmış; e-posta gönderimi test edilmiş.
- Rate limiting, hesap kilitleme, 2FA ve loglama üretim ayarları doğrulanmış.
- Health endpoint'i sır veya ayrıntılı sistem bilgisi sızdırmıyor.
- Gizli bilgiler Git geçmişinde yok; geçmişte sızdıysa rotasyon yapılmış.

11. Frontend ve API adresi

11.1 Frontend'i mevcut yerde bırakma

Frontend statik bir platformda kalacaksa üretim build'inin API base URL ayarını gerçek HTTPS API adresine çevirin. Yerel `http://localhost:8081` adresi tarayıcı kullanıcısının kendi bilgisayarını işaret eder; canlı kullanıcı için çalışmaz. Frontend'in build sırasında environment değişkenlerini gömüp gömmediğini kontrol edin; çoğu statik uygulamada değişiklikten sonra yeniden build/deploy gerekir.

```
# Örnek kontrol — framework'e göre komutlar değişir:
# 1) Üretim API base URL ayarını <FRONTEND_API_URL> olarak belirle.
# 2) Frontend production build al.
# 3) Çıktıda localhost/127.0.0.1 adresi kalmadığını kontrol et.
# 4) Deploy sonrası tarayıcı Network panelinden gerçek API çağrılarını incele.
```

11.2 Frontend'i aynı VPS'te sunma

Frontend aynı VPS üzerinde sunulacaksa reverse proxy statik dosyaları servis edebilir veya ayrı frontend konteynerine yönlendirebilir. Tek origin yaklaşımı CORS karmaşıklığını azaltabilir. Yine de API route'ları ve SPA fallback doğru yapılandırılmalı; statik dosyalar cache edilebilirken kullanıcıya özel yanıtlar cache edilmemelidir.

11.3 CORS ve HTTPS

Frontend HTTPS üzerinden açılıp API HTTP üzerinden çağrılırsa modern tarayıcılar mixed-content isteğini engeller. API de HTTPS olmalıdır. CORS origin listesine yalnız gerçek frontend adreslerini ekleyin. CORS bir kimlik doğrulama mekanizması değildir; yetkilendirme, JWT doğrulaması ve sunucu tarafı erişim kontrolleri ayrıca uygulanmalıdır.

12. Domain, DNS ve HTTPS

12.1 Alan adı henüz yokken

Domain satın alınana kadar VPS IP adresiyle ağ/servis testleri yapılabilir; ancak IP adresiyle genel amaçlı güvenilir HTTPS sertifikası almak her senaryoda mümkün olmayabilir. IP üzerinden yapılan testleri geçici kabul edin ve kullanıcı hesapları/kişisel veriyle üretim trafiğini domain ve HTTPS hazır olmadan açmayın. Alan adı daha sonra bağlandığında API base URL, CORS, callback URL'ler ve e-posta bağlantıları güncellenmelidir.

12.2 DNS planı

- Alan adını satın alırken yenileme fiyatı, transfer kilidi, DNS yönetimi ve hesap güvenliğini kontrol edin.
- A kaydını için adresine; gerekiyorsa www kaydını CNAME veya ayrı A kaydıyla yönlendirin.
- IPv6 kullanmıyorsanız yanlış AAAA kaydı bırakmayın; hatalı AAAA bazı kullanıcıların siteye erişememesine yol açabilir.
- DNS yayılımı ve kayıtları `dig`, `nslookup` veya sağlayıcı paneliyle doğrulayın.
- Domain değişikliğinde CORS allowlist, frontend build ayarı, SMTP linkleri, OAuth/redirect URL'leri ve webhook callback'lerini güncelleyin.

12.3 Nginx reverse proxy örneği

Nginx host üzerinde kurulacaksa API'nin sadece loopback portuna bind edilmesi veya host üzerinde özel olarak korunan bir bağlantı kurulması gerekir. Docker ağı üzerinden proxy konteyneri kullanmak da mümkündür. Aşağıdaki örnek yalnızca konsepti gösterir; gerçek domain, TLS dosya yolları ve proxy topolojisine göre düzenlenmelidir.

```
server {
    listen 80;
    server_name <DOMAIN> www.<DOMAIN>;

    location / {
        return 301 https://$host$request_uri;
    }
```

```
}

server {
    listen 443 ssl http2;
    server_name <DOMAIN> www.<DOMAIN>;

    # ssl_certificate ve ssl_certificate_key yollarını Certbot
    # veya seçilen proxy'nin oluşturduğu gerçek dosyalarla doldurun.

    client_max_body_size 10m;

    location / {
        proxy_pass http://api:8080;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Bu örnek, Nginx'in `api` servis adını çözebildiği aynı Docker ağı senaryosuna uygundur. Nginx host üzerinde çalışıyorsa `api:8080` DNS adı çözülmeyebilir. Ayrıca uygulamanın forwarded headers middleware'i güvenilir proxy'lerle doğru yapılandırılmalıdır; aksi halde HTTPS yönlendirmeleri, gerçek IP logları veya secure cookie davranışı yanlış olabilir.

12.4 Let's Encrypt / Certbot

Domain DNS'i doğru sunucuya yönlendikten ve port 80/443 erişilebilir olduktan sonra Let's Encrypt sertifikası alınabilir. Sertifika yenilemesi otomatik olmalı ve periyodik olarak kontrol edilmelidir.

```
# Ubuntu dokümantasyonundaki Certbot yöntemi, seçilen web sunucusuna göre değişir.
sudo snap install --classic certbot

# Nginx host üzerinde çalışıyorsa ve DNS doğrulanmışsa örnek:
sudo certbot --nginx -d <DOMAIN> -d www.<DOMAIN>

# Yenileme simülasyonu:
sudo certbot renew --dry-run
```

Nginx konteyner içindeyse host üzerindeki `--nginx` eklentisi konteynerin konfigürasyonunu otomatik düzenlemeyebilir. O durumda reverse proxy'nin kendi ACME desteğini veya webroot/DNS challenge yöntemini seçin. Sertifika ve özel anahtar dosyalarını public static klasörüne koymayın.

Resmî rehber: Ubuntu TLS sertifikaları

13. E-posta, kimlik doğrulama ve güvenlik

13.1 E-posta gönderimi

Fitness Pro'da giriş doğrulama kodu, iki faktörlü doğrulama ve hoş geldin e-postaları kullanılıyorsa SMTP üretimden test edilmelidir. SMTP sunucusu/portu, STARTTLS veya SSL modu, kullanıcı adı ve parola eşleşmeli; sağlayıcının uygulama parolası/OAuth gereksinimi kontrol edilmelidir. Gerçek kullanıcıya geçmeden önce test mailbox'ı kullanın.

13.2 JWT ve token güvenliği

- JWT signing key rastgele ve yeterli uzunlukta olmalı; repoda, logda veya frontend bundle'ında olmamalı.
- Token süreleri ve refresh token davranışı belgelenmeli; refresh token varsa rotasyon ve iptal test edilmeli.
- JWT localStorage'da tutuluyorsa XSS etkisini değerlendirin; HttpOnly/Secure/SameSite cookie yaklaşımının CSRF ve mimari etkileri ayrıca ele alınmalı.
- Claim'lerde kullanıcı ID'si, role ve issuer/audience doğrulamaları uygulamanın gerçek token yapısıyla eşleşmeli.
- Sistem saati doğru olmalı; token expiry ve e-posta kodu süreleri için UTC kullanımı tutarlı olmalı.

13.3 Rate limit, hesap kilidi ve 2FA

Mevcut iş kuralları örneğin 5/10/15 başarısız giriş eşiklerine sahipse bunların gerçek davranışı canlı ortamda test edilmelidir. Eşikler ve süreler uygulama sürümünde doğrulanmalı; IP bazlı ve hesap bazlı limitlerin birlikte kullanılması değerlendirilmelidir. Hesap kilidi, ban ve appeal kayıtları yanlış pozitiflerde kullanıcıyı kurtarabilecek idari süreçle birlikte test edilmelidir.

13.4 Kişisel veri ve loglar

Kullanıcı adı, e-posta, konum, mesajlar ve doğrulama kodları kişisel veya hassas içerik taşıyabilir. Loglara tam mesaj gövdesi, parola, JWT, 2FA kodu veya connection string yazmayın. Log saklama süresi, erişim yetkisi, yedekleme ve silme politikası belirleyin. KVKK kapsamındaki işleme, aydınlatma, veri aktarımı ve veri saklama gerekliliklerini gerektiğinde hukuk/uzman desteğiyle değerlendirin.

14. Sağlık kontrolleri, loglar ve izleme

14.1 Servis ve konteyner durumu

```
cd /opt/fitness-pro
docker compose --env-file .env -f compose.yml ps
docker compose --env-file .env -f compose.yml logs --tail=200 api
docker compose --env-file .env -f compose.yml logs --tail=200 db
docker stats --no-stream
df -h
free -h
uptime
sudo systemctl --failed
```

Loglarda sır, token, kullanıcı içeriği veya kişisel veri bulunmadığını doğrulayın. Sorun giderme sırasında `--tail` kullanın; çok büyük logları terminale dökmeyin.

14.2 HTTP health check

```
# Sunucunun kendisinde; gerçek domain/endpoint'e göre uyarlayın:
curl -fsS -D - https://<DOMAIN>/health

# DNS henüz hazır değilse, geçici olarak doğru host header / local test:
curl -fsS -D - http://127.0.0.1:<LOCAL_TEST_PORT>/health
```

Mevcut projede `/health` endpoint'i daha önce çalışmış olsa da canlı build'de aynı endpoint'in bulunduğunu ve hassas ayrıntı döndürmediğini doğrulayın. Sağlık uç noktası sadece API'nin çalışmasını değil, gerekiyorsa DB bağlantısını da güvenli biçimde kontrol eden readiness/liveness ayrımını destekleyebilir.

14.3 İzlenmesi gereken ölçümler

- CPU, RAM, swap, disk doluluğu ve disk I/O; SQL Server'ın bellek kullanımı.
- API istek sayısı, p95 yanıt süresi, 4xx/5xx oranı ve health check başarısızlıkları.
- SQL bağlantı hataları, deadlock, uzun sorgu ve bağlantı havuzu sorunları.
- Konteyner restart sayısı, log hacmi ve yedekleme başarısı.
- Sertifika son kullanma tarihi, disk doluluğu alarmı ve başarısız SSH oturumları.

Başlangıçta `docker stats`, `journalctl`, `df -h` ve sağlayıcının izleme paneli kullanılabilir. Üretim büyüdükçe metrik/uyarı sistemi ekleyin. Sadece loglara bakmak, proaktif alarmın yerini tutmaz.

15. Yedekleme ve felaket kurtarma

15.1 Önerilen yedekleme ilkeleri

- Tam veritabanı yedeğini uygulamanın veri kaybı toleransına uygun sıklıkta alın.
- Yedeği VPS dışındaki ayrı depolama hesabına aktarın; aktarımda TLS, depoda şifreleme kullanın.
- Yedekleme kimlik bilgilerini yalnız yedekleme işi için gereken en düşük yetkiyle verin.
- Yedeklerin bütünlüğünü kontrol edin ve belirli aralıklarla ayrı bir ortama geri yükleme provası yapın.
- Yedekleme başarılı/başarısız durumunu loglayın; başarısızlıkta bildirim üretin.

- RPO (kabul edilebilir veri kaybı) ve RTO (kabul edilebilir hizmet kesintisi) hedeflerini yazılı belirleyin.

15.2 Basit günlük yedekleme planı

Aşağıdaki, kavramsal bir cron örneğidir. Parolayı komut satırına gömmeyin; SQL kimlik doğrulaması ve backup komutu için erişimi kısıtlanmış güvenli bir yöntem kullanın. Yedek üretildikten sonra dosyayı şifreli biçimde VPS dışına aktarın. Bu örnek kendi başına uçtan uca yedekleme sistemi değildir.

```
# Önerilen iş akışı:
# 1. SQL Server BACKUP DATABASE komutunu çalıştır.
# 2. RESTORE VERIFYONLY ile doğrula.
# 3. Dosyanın SHA-256 değerini kaydet.
# 4. Şifreli harici depoya aktar.
# 5. Başarı/başarısızlık durumunu kaydet ve bildir.
# 6. Retention politikasına göre eski kopyaları sil.
```

Retention örneği olarak günlük yedekleri 7–14 gün, haftalık yedekleri daha uzun süre tutmak değerlendirilebilir; gerçek süre iş gereksinimleri, mevzuat, veri hassasiyeti ve depolama maliyetiyle belirlenmelidir. Silme politikasını uygulamadan önce geri yükleme ihtiyacını analiz edin.

15.3 Konfigürasyon yedeği

Compose dosyası, Dockerfile, reverse proxy yapılandırması, firewall/SSH notları ve deploy betikleri sürüm kontrolünde tutulabilir; ancak `.env`, özel anahtar, sertifika private key ve veritabanı yedeği Git'e eklenmemelidir. Konfigürasyonun sır içermeyen bir kopyasını ayrı güvenli depoda tutun.

15.4 Geri yükleme provası

En az bir kez, yeni veya izole bir ortamda şu senaryoyu deneyin: temiz SQL Server başlat, yedeği geri yükle, API'yi bağla, login/register/mesaj/profil gibi kritik akışları test et, sonuca göre süreyi ölç. Bu prova tamamlanmadan “yedeklerimiz var, güvendeyiz” varsayımı yapılmamalıdır.

16. Test planı ve kabul kriterleri

16.1 Altyapı kabul testleri

Test	Başarılı kabul ölçütü
Docker Compose	`docker compose ps` beklenen servisleri gösteriyor; restart loop yok.
SQL Server	Konteyner hazır; DB online; yedek doğrulandı; CHECKDB sonucu değerlendirildi.
API health	HTTPS üzerinden doğru status; gizli ayrıntı sızmıyor.
TLS	Sertifika geçerli; HTTP HTTPS'e yönleniyor; yenileme testi başarılı.
Firewall	80/443 ve izinli SSH dışında gereksiz portlar dışarıdan erişilemiyor.
DB erişimi	API db servis adına bağlanıyor; dışarıdan 1433 erişilemiyor.
Yedek	VPS dışı kopya mevcut; geri yükleme provası tamamlandı.

16.2 Uygulama kabul testleri

- Register: yeni test hesabı oluşturulabiliyor; beklenen rol ve varsayılan değerler doğru.
- Login: doğru/yanlış parola akışları doğru; rate limit ve hesap kilidi beklenen şekilde çalışıyor.
- E-posta doğrulama/2FA: kod e-postaya doğru geliyor; süre dolunca reddediliyor; yeniden gönderim limiti çalışıyor.
- Parola güncelleme ve token: token doğrulaması, süre sonu ve refresh davranışı doğru.
- Profil: profil verisi okunuyor/güncelleniyor; yetkisiz kullanıcı başka kullanıcı verisine erişemiyor.
- Arkadaşlık/mesaj: istek oluşturma, kabul/red ve gizlilik ayarları test ediliyor.
- Admin ve loglar: admin rol kontrolü doğru; normal kullanıcı admin uç noktalarına erişemiyor.
- Light/dark tema ve frontend: API adresi doğru; CORS/mixed content hatası yok.

- E-posta şablonları: kod, zaman ve HTML alanları doğru; içerik encode ediliyor.
- Veritabanı: kritik tablo satır sayıları ve ilişkiler kaynakla karşılaştırılıyor.

16.3 Komutla test

```
# DNS ve TLS:
nslookup <DOMAIN>
curl -I http://<DOMAIN>
curl -I https://<DOMAIN>
openssl s_client -connect <DOMAIN>:443 -servername <DOMAIN> </dev/null

# API:
curl -i https://<DOMAIN>/health

# Sunucu dinleme portları:
sudo ss -tulpn

# Docker:
docker compose --env-file .env -f compose.yml ps
docker compose --env-file .env -f compose.yml logs --since=30m api
docker compose --env-file .env -f compose.yml logs --since=30m db
```

Uç noktalar gerçek API route'larına göre değişir. Kayıt/giriş gibi endpoint'lere gerçek kullanıcı parolalarını komut satırında yazmayın; test verisi kullanın. Test tamamlandıktan sonra geçici hesapları ve verileri politikanıza uygun şekilde temizleyin.

16.4 Güvenlik doğrulaması

- Harici port taramasında SQL Server 1433 görünmüyor.
- HTTP isteği HTTPS'e yönleniyor; sertifika ve hostname doğrulaması başarılı.
- JWT veya SMTP parolası frontend bundle'ında, loglarda veya Docker image katmanlarında bulunmuyor.
- CORS yalnız onaylı origin'lere izin veriyor.
- Admin ve kullanıcı yetkileri ayrı hesaplarla test edilmiş.
- Güvenlik açığı taraması ve bağımlılık güncellemeleri tamamlanmış.
- Yedek dosyasına sadece yetkili hesap erişebiliyor; harici yedek kopyası şifreli.

17. Canlıya geçiş adım adım

17.1 Geçişten 1–3 gün önce

- Canlıya gidecek commit/tag sabitlensin; build ve bağımlılık güvenlik taraması geçsin.
- VPS ve Docker yapılandırması staging/test ortamında doğrulansın.
- Domain satın alınmışsa DNS TTL makul bir değere düşürülsün; sertifika hazırlığı yapılsın.
- Yedek ve restore provası tamamlanmış olsun; dış yedek erişimi doğrulansın.
- Kritik test senaryoları ve rollback sorumlusu belirlensin.
- Kullanıcılara bakım penceresi duyurulması gerekiyorsa duyuru yapılsın.

17.2 Geçiş anı

- Eski sistemde yazma işlemlerini durdurun veya bakım moduna alın.
- Son tam yedeği alın, checksum/verify kontrolü yapın ve VPS dışına aktarın.
- Hedef SQL Server'a geri yükleyin; DB online ve tutarlı olduğunu kontrol edin.
- API'nin hedef DB'ye bağlandığını ve migration/schema durumunun doğru olduğunu doğrulayın.
- API ve proxy'yi başlatın; health check ve kritik uygulama testlerini çalıştırın.
- DNS/API base URL yönlendirmesini değiştirin; cache ve CORS davranışını kontrol edin.
- İlk login, register, e-posta, profil, mesaj ve admin akışlarını kontrollü test edin.
- Eski sistemi geri dönüş için erişilebilir ve değişiklik yapmayacak şekilde koruyun.

17.3 İlk 24–72 saat

- API 5xx oranını, SQL hatalarını, CPU/RAM/disk ve konteyner restart'larını yakından izleyin.

- E-posta gönderimi, doğrulama kodları ve rate limit davranışını gerçek akışlarda doğrulayın.
- Yedekleme görevinin en az bir kez başarıyla çalıştığını ve dış depoya kopyaladığını kontrol edin.
- Kullanıcı bildirimi/geri bildirimlerini takip edin; kritik hatada rollback kriterini uygulayın.
- Stabilite doğrulanmadan eski sunucu ve yedekleri silmeyin.

18. Geri dönüş (rollback) planı

18.1 Rollback'i tetikleyebilecek durumlar

- API sürekli 5xx veriyor veya temel giriş/kayıt akışları çalışmıyor.
- Veritabanı geri yüklemesi eksik/hatalı ya da veri tutarlılığı şüpheli.
- Kritik yetkilendirme, kullanıcı verisi izolasyonu veya güvenlik hatası bulundu.
- E-posta doğrulama, parola güncelleme veya admin güvenlik kontrolleri çalışmıyor.
- Disk/CPU/RAM kaynakları hizmeti sürdürülemeyecek düzeyde tüketiliyor.

18.2 Geri dönüş adımları

- Yeni ortamda yazma trafiğini durdurun; yeni sisteme yazılmış verilerin nasıl korunacağını belirleyin.
- DNS veya frontend API base URL'ini eski sisteme geri yönlendirin; DNS cache nedeniyle geçişin anlık olmayabileceğini hesaba katın.
- Eski DB'nin yeni sisteme geçişten sonra yazılmadığını doğrulayın. Eski DB'yi otomatik olarak güncel kabul etmeyin.
- Yeni sistemde oluşmuş kullanıcı/veri değişiklikleri varsa kayıp olmaması için ayrıca veri uzlaştırma planı uygulayın.
- Eski ortamın sağlık kontrollerini çalıştırın ve kritik kullanıcı akışlarını test edin.
- Kök nedeni analiz edin; sorun giderilmeden tekrar geçiş yapmayın.

Rollback yalnızca DNS'i eski IP'ye çevirmek değildir. İki ortamda aynı anda yazma yapılmışsa veri çatışması ve kayıp doğabilir. Bu yüzden geçişte yazma dondurma, son yedek, veri tutarlılığı ve rollback koşulları önceden tasarlanmalıdır.

19. Sık karşılaşılan sorunlar ve teşhis

Belirti	Olası neden / kontrol
API DB'ye bağlanamıyor	Connection string'de localhost kullanımı; servis adı/ağ yanlış; DB hazır değil; login/izin veya TLS ayarı yanlış.
Konteyner restart ediyor	docker compose logs; eksik env; yanlış imaj/entrypoint; bellek/disk; izin sorunu.
HTTP 502/504	Reverse proxy upstream adı/portu yanlış; API ayakta değil; network paylaşılmıyor; timeout.
Frontend istekleri bloklanıyor	HTTP/HTTPS mixed content; CORS origin yanlış; yanlış API base URL; tarayıcı cache.
E-posta gönderilmiyor	SMTP host/port/TLS/credential; sağlayıcı kısıtı; outbound firewall; loglarda hassas bilgi olmadan hata analizi.
SQL yedeği geri yüklenmiyor	Sürüm uyumsuzluğu; dosya yolu/izin; disk dolu; mantıksal dosya adları; yedek bozuk.
Disk hızla doluyor	Docker log rotasyonu yok; eski yedekler; image katmanları; SQL log büyümesi; `docker system df` kontrolü.
TLS sertifikası alınamıyor	DNS henüz yönlenmedi; 80/443 kapalı; başka servis portu tutuyor; yanlış server_name; IPv6 kaydı hatalı.

Teşhis komutları

```
docker compose --env-file .env -f compose.yml ps
docker compose --env-file .env -f compose.yml logs --tail=250
docker inspect <CONTAINER_NAME>
docker stats --no-stream
docker system df
df -h
free -h
sudo journalctl -u docker --since "1 hour ago"
sudo ss -tulpn
sudo ufw status verbose
```

Komut çıktısını destek ekibiyle paylaşmanız gerekiyorsa IP, kullanıcı adı, token, connection string, kişisel veri ve parolaları maskeleyin.

20. Güncelleme ve bakım prosedürü

20.1 API sürüm güncellemesi

```
# CI'da build/test yapıldıktan ve immutable image tag hazırlandıktan sonra:
docker compose --env-file .env -f compose.yml pull api
docker compose --env-file .env -f compose.yml up -d --no-deps api
docker compose --env-file .env -f compose.yml ps
docker compose --env-file .env -f compose.yml logs --tail=200 api
curl -fsS https://<DOMAIN>/health
```

`` olarak commit SHA veya değişmez sürüm etiketi kullanmak, hangi sürümün çalıştığını belirlemeyi kolaylaştırır. `latest` ile üretim sürümünü izlemek yerine sürümü bilinçli yükseltin. DB schema değişiklikleri geriye uyumluluk ve rollback açısından ayrıca planlanmalıdır.

20.2 SQL Server güncellemesi

SQL Server imajını güncellemeden önce tam yedek ve restore testi alın; sürüm notlarını ve destek matrisini okuyun; staging'de geri yükleme ve sorgu testleri yapın. SQL Server veri volume'unu konteyner güncellemesi sırasında silmeyin. İmaj değişimi tek başına veritabanı formatı ve uygulama uyumluluğu garantisi vermez.

20.3 Sistem güncellemeleri

Güvenlik güncellemelerini düzenli kurun, reboot gerektiren güncellemeleri bakım penceresinde uygulayın, Docker/SQL Server/API sürüm yükseltmelerini tek seferde birbirine karıştırmayın. Her değişiklikten sonra health check, kritik akışlar, loglar ve yedekleme kontrolü yapın.

21. Maliyet kontrolü ve domain seçenekleri

Kesin fiyatlar ülke, sağlayıcı, kur, KDV, promosyon ve yenileme koşullarına göre değiştiğinden bu rehber fiyat uydurmaz. Satın alma günü sağlayıcıların fiyat sayfalarını karşılaştırın.

Kalem	Düşük maliyetli yaklaşım	Dikkat edilmesi gereken
Linux VPS	Aylık, büyütülebilir başlangıç paketi	RAM/CPU/disk ve trafik sınırları; yedek/snapshot ücreti
Domain	İhtiyaç duyulan tek alan adı ve sade DNS	İlk yıl promosyonu yerine yenileme ücreti; transfer/hesap güvenliği
TLS	Let's Encrypt ile ücretsiz sertifika	Otomatik yenileme ve DNS/port koşulları
Yedekleme	VPS dışı şifreli nesne depolama veya ayrı sunucu	Egress, depolama, API ve kurtarma maliyeti
E-posta	Mevcut güvenilir SMTP sağlayıcısı veya uygun plan	Günlük limit, teslim edilebilirlik, SPF/DKIM/DMARC
SQL Server	Lisansı ve özellik ihtiyacını doğrulanmış edition	Üretim lisansı; Developer yalnız geliştirme/test

Domain satın alındığında DNS yönetimi için mevcut registrar'ın DNS'i veya güvenilir bir DNS sağlayıcısı kullanılabilir. CDN/WAF gibi ek hizmetler faydalı olabilir ama ilk aşamada gereksiz karmaşıklık ve maliyet yaratmaması için ihtiyaç ve risk üzerinden karar verilmelidir.

22. Canlıya geçiş kontrol listesi

- [ ] Canlıya gidecek commit/tag belirli ve build başarılı.
- [ ] Bağımlılık güvenlik taraması tamam; kritik/yüksek bulgular değerlendirilmiş.
- [ ] VPS güncel; SSH anahtarlı; root/password login kapalı veya gerekçeli istisna belgeli.
- [ ] Firewall: yalnız gereken portlar açık; SQL 1433 dışarı kapalı.
- [ ] Docker Engine/Compose resmi kaynaktan kurulu ve güncel.
- [ ] SQL Server imajı/sürümü/lisansı doğrulandı; kalıcı volume mevcut.
- [ ] `.env`/secrets Git dışı; izinler kısıtlı; sırlar benzersiz.
- [ ] Veritabanı yedeği alındı, checksum ve restore doğrulaması yapıldı.
- [ ] API connection string Docker içindeki doğru DB servisine bağlanıyor.
- [ ] API portu internete doğrudan açık değil; reverse proxy üzerinden erişiliyor.
- [ ] Domain DNS'i doğru; HTTPS geçerli; sertifika yenileme testi yapıldı.
- [ ] Frontend production API URL'si doğru; CORS ve mixed-content sorunu yok.
- [ ] Login/register/2FA/e-posta/profil/mesaj/admin senaryoları test edildi.
- [ ] Loglar sır ve kişisel veri sızdırmıyor; log rotasyonu açık.
- [ ] VPS dışı yedekleme ve başarısızlık bildirimi doğrulandı.
- [ ] Geri dönüş adımları, sorumlu kişi ve tetikleme koşulları yazılı.
- [ ] Eski sistem stabilite süresi bitmeden silinmeyecek.

23. Resmî kaynaklar

Kurulum ve güvenlik adımlarını uygulama günü resmi belgelerin güncel sürümleriyle karşılaştırın. Bu bağlantılar genel rehber niteliğindedir; sağlayıcı fiyatları ve ürün destek matrisleri ayrıca doğrulanmalıdır.

Docker Engine kurulum belgeleri: https://docs.docker.com/engine/install/ — Linux dağıtımına göre Docker Engine kurulum adımları.

Docker ve firewall: https://docs.docker.com/engine/network/packet-filtering-firewalls/ — Docker'ın iptables/DOCKER-USER ve port yayınlama davranışı.

.NET Docker belgeleri: https://learn.microsoft.com/dotnet/core/docker/ — .NET uygulamalarını Docker ile derleme ve çalıştırma.

SQL Server Linux konteynerleri: https://learn.microsoft.com/sql/linux/quickstart-install-connect-docker — SQL Server konteynerini kurma ve bağlanma.

SQL Server backup/restore: https://learn.microsoft.com/sql/relational-databases/backup-restore/ — SQL Server veritabanı yedekleme ve geri yükleme.

Ubuntu TLS sertifikaları: https://ubuntu.com/server/docs/how-to/security/obtain-tls-certificates/ — Let's Encrypt ve Certbot ile TLS.

Ubuntu firewall: https://ubuntu.com/server/docs/firewalls/ — UFW ve temel firewall yönetimi.

Ubuntu OpenSSH: https://ubuntu.com/server/docs/how-to/security/openssh-server/ — OpenSSH kurulum ve yapılandırma.

Microsoft SQL Server lisanslama: https://www.microsoft.com/licensing/product-licensing/sql-server — Üretim lisans kararını doğrulamak için resmi lisans bilgileri.

24. Tam uygulama için kapsamlı ana prompt

Aşağıdaki prompt, bir kod asistanına veya teknik uygulama ekibine verilecek şekilde hazırlanmıştır. Projeye erişimi olmayan araçların dosyaları görüyormuş gibi davranmaması, mevcut kodu incelemeden varsayım yapmaması ve yıkıcı işlemlerden önce açık onay istemesi özellikle belirtilmiştir.

ROL VE AMAÇ
Fitness Pro projesinin mevcut mimarisini koruyarak Linux VPS üzerinde güvenli, yedeklenebilir, izlenebilir ve geri alınabilir bir üretim dağıtımına taşınması için kıdemli .NET, SQL Server, Docker, Linux ve uygulama güvenliği mühendisi gibi çalış. Amaç, yalnızca teorik bir plan yazmak değil; erişimin olan dosyaları incelemek, kanıtla desteklenen bir uygulama planı oluşturmak, gerekli yapılandırmaları dikkatli biçimde hazırlamak ve doğrulama sonuçlarını raporlamaktır.

MEVCUT BAĞLAM
- Backend: C# / ASP.NET Core .NET 10 API.
- Veritabanı: Microsoft SQL Server; üretim veritabanı adı, SQL Server sürümü, collation, boyut ve login bilgileri önce gerçek yapılandırmadan doğrulanmalıdır.
- Dağıtım hedefi: Linux VPS + Docker Engine + Docker Compose.
- API ve SQL Server aynı VPS üzerinde ayrı konteynerlerde çalışacaktır.
- SQL Server yalnızca özel Docker ağı üzerinden erişilebilir olmalı; host üzerinde 1433 portunu internete yayımlama.
- API, reverse proxy arkasında çalışmalı; API'nin iç portunu doğrudan internete açma.
- Frontend mevcut mimari korunarak ya mevcut statik barındırmada kalmalı ya da kontrollü şekilde aynı VPS üzerinde servis edilmelidir. Hangi seçeneğin mevcut projeye uyduğunu incele ve gerekçelendir.
- Domain henüz satın alınmamıştır. Önce IP ile sınırlı teknik doğrulama yapılabilir; gerçek kullanıcı trafiği için domain, HTTPS, doğru CORS ve üretim API URL'si tamamlanmalıdır.
- Maliyet düşük tutulmalı; ancak SQL Server lisansı, VPS kaynakları, harici yedekleme, alan adı ve e-posta gönderimi maliyetlerini gizleme veya yok sayma.
- Bilinmeyen hiçbir parolayı, IP'yi, alan adını, DB adını, image tag'ini veya sağlayıcı fiyatını uydurma. Bilinmeyenleri <PLACEHOLDER> olarak işaretle ve hangi kaynaktan doğrulanacağını açıkla.

KESİN ÇALIŞMA KURALLARI
1. Önce depodaki dosyaları, .csproj, Dockerfile, compose dosyaları, appsettings, health endpoint'leri, connection string anahtarları, CORS ayarları, authentication, JWT, e-posta/SMTP ve loglama kodunu incele. Dosyaya erişimin yoksa erişimin varmış gibi davranma; kullanıcıdan dosya istediğini söyle veya yalnızca genel plan üret.
2. Mevcut davranışı koru. İstenen taşımayla ilgisiz refactor, yeni UI, yeni framework veya büyük mimari değişiklik yapma.
3. Her öneriyi mevcut dosya ve gerçek yapılandırma bulgusuyla ilişkilendir. Varsayımları ayrı işaretle.
4. Herhangi bir yıkıcı komut, DB overwrite/drop, volume silme, mevcut veriyi değiştirme, production secret rotasyonu veya DNS geçişinden önce açık onay iste. Otomatik olarak yıkıcı işlem çalıştırma.
5. Gizli değerleri çıktılarda gösterme. .env, connection string, JWT key, SMTP password, private key ve backup dosyalarını Git'e ekleme. Eğer bir sır daha önce commit edilmişse yalnız dosyadan silmeyi yeterli sayma; rotasyon planı hazırla.
6. Komutları çalıştırmadan önce hedef işletim sistemi, shell ve çalışma dizinini belirt. PowerShell ile Bash komutlarını karıştırma. Komutun beklenen sonucunu ve başarısızlıkta teşhis adımını yaz.
7. Güncel sürüm, destek matrisi, lisans veya güvenlik bilgisi gerekiyorsa resmi kaynaklardan doğrula; tarih ve kaynak bağlantısını ekle. Güncel fiyat uydurma.
8. Çalıştırılmamış testi başarılı olarak raporlama. Yapılan, yapılmayan, engellenen ve kullanıcı onayı bekleyen işleri ayrı listele.
9. Proje .NET 10 kullandığı için Docker SDK/runtime imajlarını .NET 10 ile uyumlu seç; imaj etiketlerini resmi belgelerle doğrula. Production için SQL Server Developer Edition kullanma; lisans/edition kararını kullanıcıya açıkça bildir.
10. İşlemleri küçük ve geri alınabilir adımlara böl. Her kritik aşamadan sonra kontrol noktası ekle.

AŞAMA 1 — ENVANTER VE RİSK ANALİZİ
- Repository kökünü ve gerçek backend/frontend dizinlerini belirle.
- git status, son commit, solution/project yapısı, mevcut Dockerfile ve compose dosyalarını incele.
- Projenin gerçek DB connection string anahtarını ve runtime'da nasıl okunduğunu bul.
- Gerçek API portu, ASPNETCORE_URLS, health endpoint'i, CORS origin listesi, JWT config, SMTP config, rate limit, 2FA ve loglama ayarlarını tespit et.
- Veritabanı adını, sürümünü, boyutunu, tabloları, login/user izinlerini, collation ve compatibility level değerlerini doğrulama adımlarını yaz.
- Çalışan geliştirme ortamı ile hedef production ortamı arasındaki farkları tablo halinde çıkar.
- Bilinmeyenleri uydurma; doğrulanması gerekenler listesi oluştur.
- Kritik riskleri olasılık/etki/önlem tablosunda göster.

AŞAMA 2 — HEDEF MİMARİ
- Linux VPS, Docker Engine/Compose, .NET 10 API, SQL Server konteyneri, reverse proxy ve frontend için açık mimari çiz.
- SQL Server'ın 1433 portunu host'a publish etme; yalnız özel Docker ağı kullan.
- API'yi reverse proxy arkasına al; yalnız 80/443 internetten açık olsun. SSH'yi mümkünse yönetim IP'siyle sınırla.
- Frontend'in mevcut barındırma yerinde kalması ve aynı VPS'te servis edilmesi seçeneklerini karşılaştır; en az değişiklikle mevcut mimariyi koruyan seçeneği öner.
- Tek VPS'in tek arıza noktası olduğunu belirt; VPS dışı yedek ve restore provası şartı koy.
- Domain alınana kadar geçici IP testlerinin sınırlarını açıkla.

AŞAMA 3 — VPS VE HOST GÜVENLİĞİ
- Desteklenen Linux LTS/Stable sürümünü seçmek için resmi Docker ve SQL Server belgelerini kontrol et.
- Sağlayıcı seçimini kaynak, disk, trafik, snapshot, kurtarma konsolu, lokasyon ve toplam maliyet kriterleriyle yap; fiyat uydurma.
- SSH key ile yönetici hesabı, sudo, root/password login kapatma, apt güncellemesi, unattended security updates, fail2ban ve saat senkronizasyonunu adım adım tarif et.
- SSH yapılandırması değiştirilmeden önce ikinci oturum ve sağlayıcı konsolunu doğrula.
- UFW ve sağlayıcı firewall kurallarını birlikte değerlendir. Docker'ın port yayınlamasının UFW'yi aşabilme ihtimalini resmi dokümantasyonla açıkla.
- Sadece gerekli portları aç; SQL Server, Docker remote API ve API iç portunu internetten kapalı tut.
- Kurulum ve doğrulama komutlarını ayrı ver.

AŞAMA 4 — DOCKER VE DAĞITIM DOSYALARI
- Resmi depodan Docker Engine ve Compose plugin kurulumunu güncel resmi talimatlarla hazırla.
- Var olan compose dosyasını silmeden önce incele; gerekiyorsa ayrı production compose dosyası öner.
- API için çok aşamalı .NET 10 Dockerfile oluştur veya mevcut olanı güvenli biçimde düzelt. Gerçek csproj yolu ve assembly adı doğrulanmadan tahmin etme.
- .dockerignore içine .git, bin/obj, node_modules, .env, private key, sertifika ve backup gibi uygun kalıpları ekle.
- SQL Server imajını test edilmiş sürüme sabitle; belirsiz latest etiketi kullanma.
- API ve DB için restart policy, health/readiness stratejisi, kalıcı volume, özel network, log rotasyonu ve uygun kaynak sınırlarını tasarla.
- SQL Server için host port mapping ekleme.
- Secret'ları image katmanlarına kopyalama; .env/secret yönetimini ve dosya izinlerini açıkla.
- `docker compose config`, `pull`, `build`, `up`, `ps`, `logs` ve rollback komutlarını sun.
- Her yapılandırma örneğinde gerçek değerleri placeholder olarak bırak; sırları çıktı olarak yazdırma.

AŞAMA 5 — SQL SERVER VE VERİ TAŞIMA
- Kaynak DB için bakım penceresi, yazma dondurma, son full backup, checksum, RESTORE VERIFYONLY ve SHA-256 kontrolünü tarif et.
- Backup dosyasını güvenli şekilde VPS'e aktar; dosya boyutu ve hash eşleşmesini kontrol et.
- SQL Server konteynerinin backup dosyasını görebileceği yolu açıkça kur; host path ile container path farkını belirt.
- RESTORE FILELISTONLY çıktısından logical data/log isimlerini al; isimleri tahmin etme.
- RESTORE DATABASE için MOVE hedeflerini gerçek data/log konumuna göre hazırla.
- SQL Server sürüm/edition uyumluluğu, backup compression desteği, collation, compatibility level ve lisans şartlarını kontrol et.
- DBCC CHECKDB, online state, kritik tablo/row count ve uygulama sorgularıyla doğrulama yap.
- Server-level login'lerin backup ile otomatik taşınmış varsayımını yapma; API için özel least-privilege login/user oluşturma planı yaz.
- `sa` hesabını API'de kullanma; db_owner yetkisini gerekçesiz verme.
- Hedef DB'yi overwrite/drop edecek komutları onaysız çalıştırma.
- En az bir izole ortamda gerçek restore provası yapılmadan yedeklemeyi tamamlanmış sayma.

AŞAMA 6 — API VE FRONTEND KONFİGÜRASYONU
- Connection string'de localhost/LocalDB kullanımını tespit et; API konteynerinden DB servis adı ve portuyla bağlanacak şekilde ayarla.
- ASP.NET Core environment variable eşlemesini, özellikle ConnectionStrings__DefaultConnection davranışını gerçek projeye göre doğrula.
- Production environment, HTTPS/forwarded headers, CORS allowlist, JWT signing key, refresh token, SMTP, 2FA, rate limit ve loglama ayarlarını denetle.
- API'nin health endpoint'inin üretim build'inde gerçekten mevcut olduğunu doğrula.
- Frontend production API base URL'sini gerçek HTTPS URL'ye geçir; build sırasında gömülen config varsa rebuild gerektiğini belirt.
- HTTP/HTTPS mixed content, CORS ve tarayıcı Network paneli testlerini açıkla.
- Frontend bundle içinde secret olmadığını kontrol et; public API key ile server secret farkını ayır.
- E-posta doğrulama, login, register, password update, profile, friend request, messaging, admin access ve privacy flows için test planı üret.

AŞAMA 7 — DOMAIN VE TLS
- Domain satın alınmadıysa DNS ve HTTPS aşamasını beklemeye al; geçici IP testinin güvenli sınırlarını belirt.
- Domain alınca A/AAAA/CNAME kayıtlarını, yanlış AAAA riskini, DNS çözümlemesini, TTL'yi ve frontend/CORS/redirect ayarlarını güncelle.
- Nginx/Caddy/Traefik seçeneklerinden mevcut mimariye en az müdahale edenini öner.
- Let's Encrypt sertifikası ve otomatik yenilemeyi kur; domain ve 80/443 erişim önkoşullarını belirt.
- Reverse proxy'de forwarded headers ve gerçek client IP güvenini doğru ayarla.
- Sertifika yenileme dry-run ve HTTPS doğrulama komutlarını ver.
- SQL/API portlarını doğrudan public yapma.

AŞAMA 8 — YEDEKLEME, İZLEME VE BAKIM
- DB full backup sıklığı, retention, şifreleme, VPS dışına kopya, hash/verify, hata bildirimi ve restore provası için uygulanabilir prosedür oluştur.
- RPO/RTO hedefleri belirlenmesi gerektiğini açıkla; değerleri kullanıcı gereksinimi olmadan kesin kabul etme.
- Docker log rotasyonu, disk doluluğu, CPU/RAM, SQL connection error, API 5xx, certificate expiry ve backup failure için

izleme öner.
- Konfigürasyon yedeğini sır içermeyecek şekilde sakla.
- API ve SQL Server güncelleme prosedürlerini birbirinden ayır; önce staging ve backup/restore testi yap.
- Yedeklerin tek VPS'te kalmamasını sağla.

AŞAMA 9 — TEST, CANLIYA GEÇİŞ VE ROLLBACK
- Altyapı, TLS, firewall, DB, API health, frontend, auth, email, 2FA, messaging, profile, admin, privacy ve log güvenliği için kabul kriterli test tablosu oluştur.
- Taşıma öncesi, geçiş anı ve ilk 24–72 saat adımlarını sırala.
- Son yedek, yazma dondurma, DB restore, API smoke test, DNS geçişi ve eski ortamı koruma adımlarını uygula.
- Rollback tetikleme kriterleri, veri çatışması riskleri, DNS cache ve eski DB'nin güncel olmayabileceği durumları açıkla.
- Rollback yalnız DNS değişimi değildir; yeni ortamda oluşan yazmaları koruma/uzlaştırma planı gerekir.
- Her testte çalıştırılacak komutu, beklenen sonucu, başarısızlık teşhisini ve geri alma adımını yaz.
- Çalıştırılmamış testleri tamamlandı olarak işaretleme.

AŞAMA 10 — TESLİMAT
- Eksiksiz bir uygulama rehberi ve gerekiyorsa değiştirilecek dosyaların tam içeriğini üret.
- Değişen dosyaları ve neden değiştiğini listele.
- Yapılan değişiklikler, çalıştırılan komutlar, gerçek test sonuçları, kalan işler ve kullanıcı onayı bekleyen adımları ayrı raporla.
- PDF mümkünse PDF oluştur; değilse tam metni ver ve sınırlamayı açıkça bildir.
- En sonda canlıya geçiş checklist'i ve geri dönüş planı yer alsın.
- Gerçek IP, domain, parola, DB adı, fiyat veya test sonucu uydurma.

Belgenin sonu. Canlıya geçiş, gerçek ortam ve test sonuçları doğrulanmadan tamamlanmış sayılmamalıdır.